

Autonomous Lane Following and RL-Based Obstacle Avoidance

on a DB21J Duckiebot

**Chitsidzo Varaidzo Nemazuwa Kanaya
Nisa Ozora**

March 24, 2026



TUM Uhrenturm

Outline

- 1 Introduction to Duckietown
- 2 System Architecture: FSM and RL Concept
- 3 How TD3 Works
- 4 Packages Added and Changed
- 5 Challenges Encountered
- 6 Future Implementations
- 7 Demo

What is Duckietown?

- Open-source robotics education and research platform
- Miniaturized urban environment with standardized road segments
- Robot: **DB21J Duckiebot** — differential drive, monocular camera, wheel encoders, ToF sensor
- Software stack: **ROS Noetic** on **daffy** branch

Standard Pipeline

Camera



Line Detector



Ground Projection



Lane Filter (d, ϕ)



Lane Controller

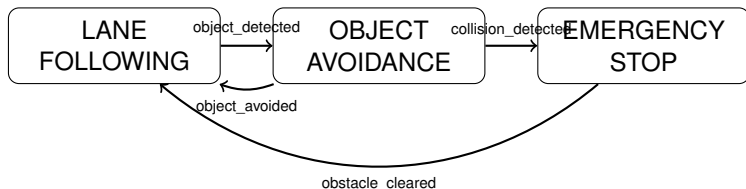
Project Goal

- Extend the standard lane following pipeline with **RL-based obstacle avoidance**
- Use a **Finite State Machine** to govern high-level behavior switching
- Train a **TD3 agent** directly on the physical robot
- Deploy trained weights for real-time inference

Outline

- 1 Introduction to Duckietown
- 2 System Architecture: FSM and RL Concept**
- 3 How TD3 Works
- 4 Packages Added and Changed
- 5 Challenges Encountered
- 6 Future Implementations
- 7 Demo

Finite State Machine Overview (Deployment)



LANE_FOLLOWING

Standard pipeline active.
Drives straight through intersections.

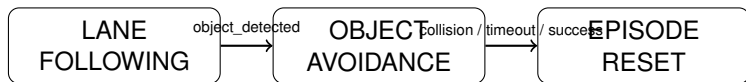
OBJECT_AVOIDANCE

TD3 agent takes over. Sends waypoints to avoider controller.

EMERGENCY_STOP

Bot halts. Waits for ToF to confirm obstacle cleared.

Finite State Machine Overview (Training)



LANE_FOLLOWING

Initial state. Robot follows lane until object is detected.

OBJECT_AVOIDANCE

RL agent attempts avoidance during training.

EPISODE_RESET

Episode ends. Robot stops and waits for manual reset.

RL Concept: State, Action, Reward

State Space (dim=2)

$s = [\text{tof_distance}, \text{lane_offset}]$

Action Space (dim=1)

$a = [\omega]$ — continuous, clipped to $[-1, 1]$

Reward Function

Condition	Reward
Collision ($d \leq 0.05\text{m}$)	-10
Critical ($d < 0.20\text{m}$)	-1
Safe ($d > 0.20\text{m}$)	+10
Obstacle avoided	+10
Timeout	-5

Outline

- 1 Introduction to Duckietown
- 2 System Architecture: FSM and RL Concept
- 3 How TD3 Works**
- 4 Packages Added and Changed
- 5 Challenges Encountered
- 6 Future Implementations
- 7 Demo

Twin Delayed DDPG (TD3)

- Model-free, off-policy algorithm for **continuous action spaces**
- Actor NN — decides what action to take
- Critic NN — evaluates action made by actor
- Improves DDPG by addressing **overestimation bias**
- **Twin Critics** $Q_{\theta_1}, Q_{\theta_2}$ — use minimum Q value to reduce overestimation
- **Replay Buffer** — a data structure where past experiences are stored, allowing the agent to learn from its history → increase speed of learning and reduced correlation between experiences

Training Loop (per timestep)

1. Observe state s
2. Select action $a = \pi(s) + \epsilon$
3. Execute, observe r, s'
4. Store (s, a, r, s') in replay buffer
5. Sample mini-batch, update critics
6. Every 2 steps: update actor
7. Soft update target networks

Training vs Deployment

Training Node

- Exploration noise $\epsilon \sim \mathcal{N}(0, 0.1)$
- Random actions for first 20 timesteps (so the robot learns after 20 timesteps)
- Replay buffer
- Saves weights to `/data/safe_rl_weights/` after reaching a maximum of 30 episodes
- Saves after every episode to avoid issues with losing progress when training crashes

Deployment Node

- Loads pre-trained `actor.pth`
- Inference only — no replay buffer, no training
- Detects collision and publishes `collision_detected`
- Returns to lane following on success

Outline

- 1 Introduction to Duckietown
- 2 System Architecture: FSM and RL Concept
- 3 How TD3 Works
- 4 Packages Added and Changed**
- 5 Challenges Encountered
- 6 Future Implementations
- 7 Demo

New and Modified Packages

New Packages

- `safe_rl` — TD3 training and deployment (training + inference nodes)
- `our_fsm` — custom FSM configs for deployment and training (incl. `EPISODE_RESET`)

Modified Packages

- `obstacle_detection` — ported ToF node to daffy; added invalid reading filter
- `avoider_controller_node` — fixed waypoint checking logic
- `master.launch` — disabled anti-instagram; direct camera remap

Key Hardware Insight

Anti-instagram introduced multi-second delays, causing the lane filter to use stale images. Disabling it eliminated zigzag behavior.

Avoider Controller Fix

Problem: Incorrect Waypoint Checks

- Controller advanced too early for first waypoints
- Only X-direction considered, ignoring Y deviation
- Final waypoint threshold too strict (0.05 m)
- Result: robot missed trajectory and never published `avoidance_done`

Fix: Correct Distance Logic

- Non-final: use Euclidean distance
 $\|p_c - p_t\| < 0.05$
- Final: check $|\Delta y| < 0.15$ (loosened threshold)
- Ensures full trajectory execution
- `avoidance_done` now reliably triggered

Outline

- 1 Introduction to Duckietown
- 2 System Architecture: FSM and RL Concept
- 3 How TD3 Works
- 4 Packages Added and Changed
- 5 Challenges Encountered**
- 6 Future Implementations
- 7 Demo

Challenges Encountered

- Stack instability (daffy vs ente compatibility)
- Avoidance completion not detected → fixed via waypoint logic
- Noisy ToF readings → false obstacle triggers

Outline

- 1 Introduction to Duckietown
- 2 System Architecture: FSM and RL Concept
- 3 How TD3 Works
- 4 Packages Added and Changed
- 5 Challenges Encountered
- 6 Future Implementations**
- 7 Demo

Future Work

- **Using Object Classification** - Use object classification to truly identify if an object is in front of us instead of solely relying on the ToF obstacle detection
- **Zonotope Safety Shields** — formal reachability-based safety layer using the library zonopy; provides provable collision avoidance guarantees
- **Richer State Space** — add linear velocity, action history, and ToF reading history to improve anticipation
- **Weight Resumption** — load weights at training startup to enable multi-session training across bot restarts

Outline

- 1 Introduction to Duckietown
- 2 System Architecture: FSM and RL Concept
- 3 How TD3 Works
- 4 Packages Added and Changed
- 5 Challenges Encountered
- 6 Future Implementations
- 7 Demo**



Conclusion

- Successfully integrated RL into Duckietown pipeline
- Achieved real-time obstacle avoidance on physical robot
- Could improve obstacle detection and add zonotopes for better deployment and safety checking
- Foundation for safer autonomous navigation

Thank you for your attention!

Repository	<code>github.com/nayachewsudon/duckietown-btown</code>
Weights	<code>/data/safe_rl_weights/</code> on the bot
Stack	ROS Noetic, daffy, DB21J
